

Answer key — finished scripts and challenge solutions.

Welcome to Scratch G - Space Invaders. This is the capstone: a full arcade game. It pulls together everything - movement, variables, conditionals, custom blocks, and **broadcast** - and adds the one idea that makes big Scratch projects possible: **clone coordination**. A single **Invader** sprite stamps out 55 clones, and a few shared variables plus a broadcast *beat* make the whole fleet march, drop, and turn as one. The rule to say out loud: *if there are lots of them on screen, they are clones*. We build in small steps: after *every* project you press the green flag and the game runs - half-built on purpose. You will see each problem before you fix it. Open scratch.mit.edu, click *Create*, and build the invasion. Your teacher will give you the costume images.

The block groups you will use. Each block has a colour. The colour tells you which group it is in. Find the colour first, then find the block.

Motion	(blue)	Move the sprite: move, turn, go to, point towards, change x or y.
Looks	(purple)	Change how the sprite looks: say, costume, size, backdrop.
Sound	(pink)	Play a sound.
Events	(yellow)	Start a script (green flag, key, click, when I receive) and broadcast messages.
Control	(orange)	Choose when and how often blocks run: wait, forever, repeat, if, stop.
Sensing	(light blue)	Ask about the world: touching?, key pressed?, mouse, ask and answer.
Operators	(green)	Do maths and comparisons: plus, pick random, less-than, equals, more-than, and/or.
Variables	(dark orange)	Remember a value like score: set, change, and show a variable.

Project 1: The ship

Drive the player ship left and right along the bottom row. First make it move at all - then fix the problem you will see with your own eyes: it slides right off the stage.

Get ready. Add a **Player** sprite and upload `ship_fighter_1` as its costume. The ship lives on one row near the bottom (`y = -150`).

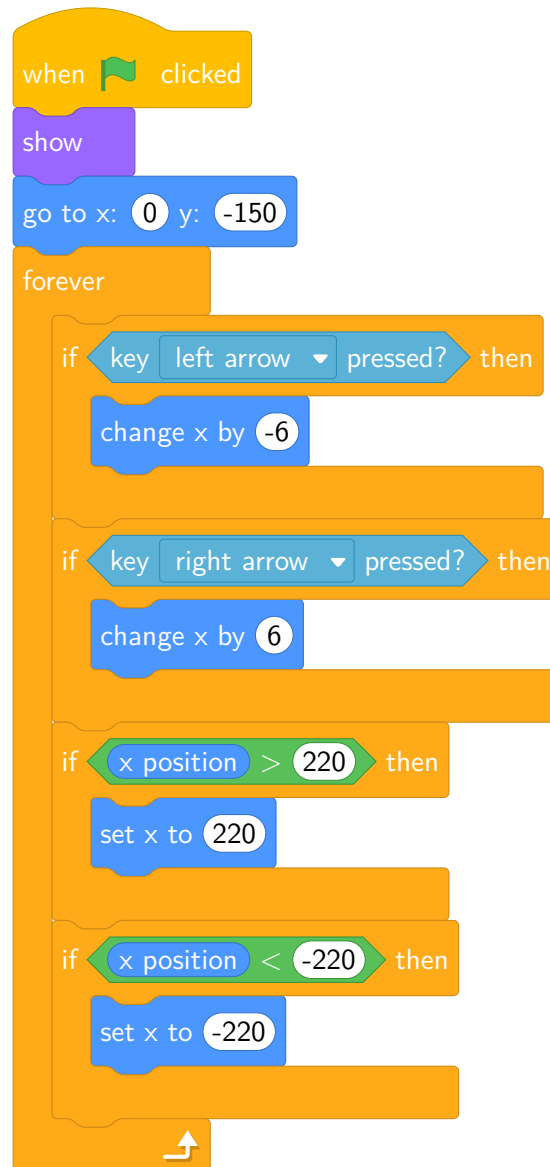
New blocks.

<code>if <key () pressed?></code>	Reading the keys inside one forever loop moves the ship smoothly, in either direction.
<code>set x to ()</code>	A guard: clamp <code>x position</code> back to the edge so the ship can't slide off the stage.

Build it.

1. On the **Player**: when green flag clicked, show, go to x: (0) y: (-150).
2. Add a **forever** loop. Inside it, two key checks: if <key [left arrow] pressed?> then change x by (-6), and the same for **right arrow** with change x by (6).
3. **Checkpoint** - green flag: drive with the arrow keys. It works! Now hold a key down... and watch the ship squash itself into the edge. That is the problem we fix next.
4. Add two guards to the loop: if <(x position) > (220)> then set x to (220), and if <(x position) < (-220)> then set x to (-220).
5. Green flag again: the ship now stops cleanly at each wall.

When you are done, your code should look like this:



Player: drive left/right, stay on screen

Reading keys with `if <key () pressed?>` inside a forever loop (not separate `when key pressed` hats) is what makes the ship feel responsive - the same trick Maze Quest used.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Tune the feel: if the ship is too slow, raise both moves to (8); too twitchy, lower them to (4).
Expected: The ship's speed matches your taste.
 2. Swap the look: try `ship_delta_1` as the costume instead.
Expected: The classic arrow-fighter shape.
-

Project 2: Fire!

The ship drives - now make it shoot. Press space and a laser clone flies up from the ship. This version fires as fast as you can mash - a spray. That is fine for now; seeing the spray IS the point.

Get ready. Add a **Bullet** sprite (costume `bullet_cyan_1`). Start the sprite hidden - only its clones ever show.

New blocks.

<code>go to [Player]</code>	Jump the bullet to the ship's muzzle before cloning, so the shot starts where the ship is.
<code>create clone of [myself]</code>	Spawn a copy that runs the <code>when I start as a clone</code> script - your flying laser.
<code>delete this clone</code>	A clone's own life: it runs the clone hat, then cleans itself up when the shot is spent.

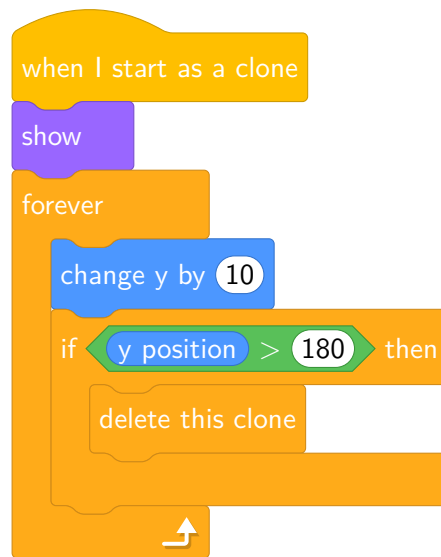
Build it.

1. On the **Bullet**: `when green flag clicked`, `hide`.
2. Add `when [space] key pressed`: `go to [Player]`, `create clone of [myself]`, `start sound [shoot]`.
3. Add `when I start as a clone`: `show`, then a `forever` that does `change y by (10)`.
4. **Checkpoint** - green flag: press space. A laser flies up from the ship - and off the top it keeps flying, invisible, forever. Every shot is a clone that never dies. Fix that:
5. In the clone's `forever`, add `if <(y position) > (180)>` then `delete this clone`.
6. Green flag: each shot now vanishes at the top. But hold space down - a wall of lasers! The 1978 arcade allowed only ONE shot at a time. That is the next project.

The trigger (left) and each laser's flight (right):



Bullet: fire on space (no limit - yet)



Bullet: fly up, vanish off the top

A clone that flies off the top does not disappear - Scratch keeps it alive (and counted) until `delete this clone` runs. Spraying hundreds of immortal clones is how projects grind to a halt.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Colour your shot: swap the costume to `bullet_orange_1` or `bullet_green_1`.

Expected: The laser matches your theme.

2. Faster laser: change `change y by (10)` to `(16)`.

Expected: The shot crosses the screen noticeably quicker.

Project 3: One shot at a time

Shooting works - too well. Bring in the 1978 rule: only one laser on screen. A state flag `bulletActive` gates the trigger, exactly like the Dino's `grounded?` gated jumping.

Get ready. Make a variable `bulletActive` For all sprites.

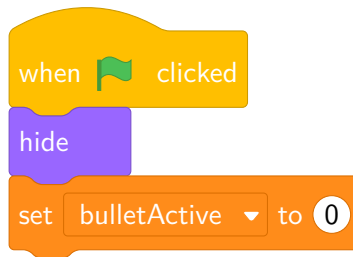
New blocks.

```
if <(bulletActive) = (0)> The gate: fire only when no laser is flying. Set it to 1 when you shoot, back to 0 when the shot is spent.
```

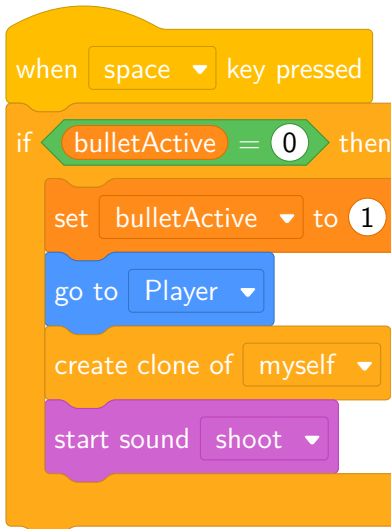
Build it.

1. In the Bullet's `when green flag clicked` script, add `set [bulletActive] to (0)`.
2. In the `when [space] key pressed` script, wrap the three blocks in `if <(bulletActive) = (0)>`, and add `set [bulletActive] to (1)` as the first block inside.
3. In the clone's off-the-top check, before `delete this clone`, add `set [bulletActive] to (0)` - the shot is spent, so reload.
4. **Checkpoint** - green flag: hold space down. One laser at a time; the next fires only after this one leaves the top.

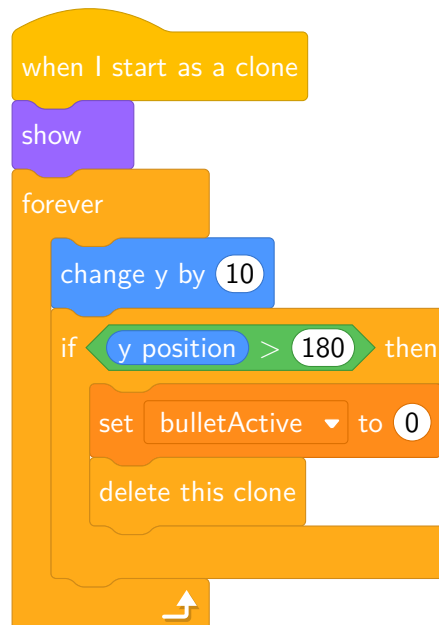
All three Bullet scripts, finished:



Bullet: start hidden, gate open



Bullet: fire, but only if none is flying



Bullet: fly up, then reload the gate

`bulletActive` is a state flag: a 0/1 variable that gates an action so it can only happen when it should. Every set must have a matching reset - forget the reset and you can never fire again.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Machine-gun experiment: pull the three blocks OUT of the `if` wrapper, hold space, watch the spray - then put them back.

Expected: You feel exactly what the flag buys you.

Project 4: Build the fleet

Something to shoot at! Stamp out the classic $5 \times 11 = 55$ invaders from ONE sprite, growing it in three checkpoints: one invader, then a row, then the grid. The tool is a custom block `spawn invader (x) (y)`.

Get ready. Add an **Invader** sprite with two wiggle costumes, e.g. `invader_green_1` and `invader_green_2`. Make variables `invadersLeft`, `row`, `col` (all *For all sprites*) and, *For this sprite only*, `me`.

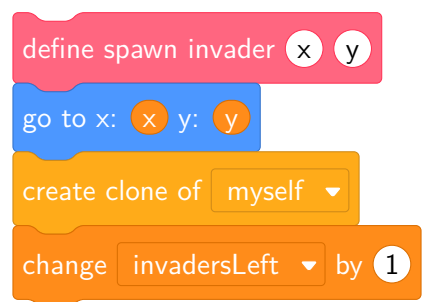
New blocks.

<code>define spawn invader (x) (y)</code>	Make it under <i>My Blocks</i> -> <i>Make a Block</i> ; add two number inputs named <code>x</code> and <code>y</code> . The crimson header is where the block's own steps go.
<code>go to x: (x) y: (y)</code>	Position the original at the target spot; the clone is born THERE, so no pile-up.

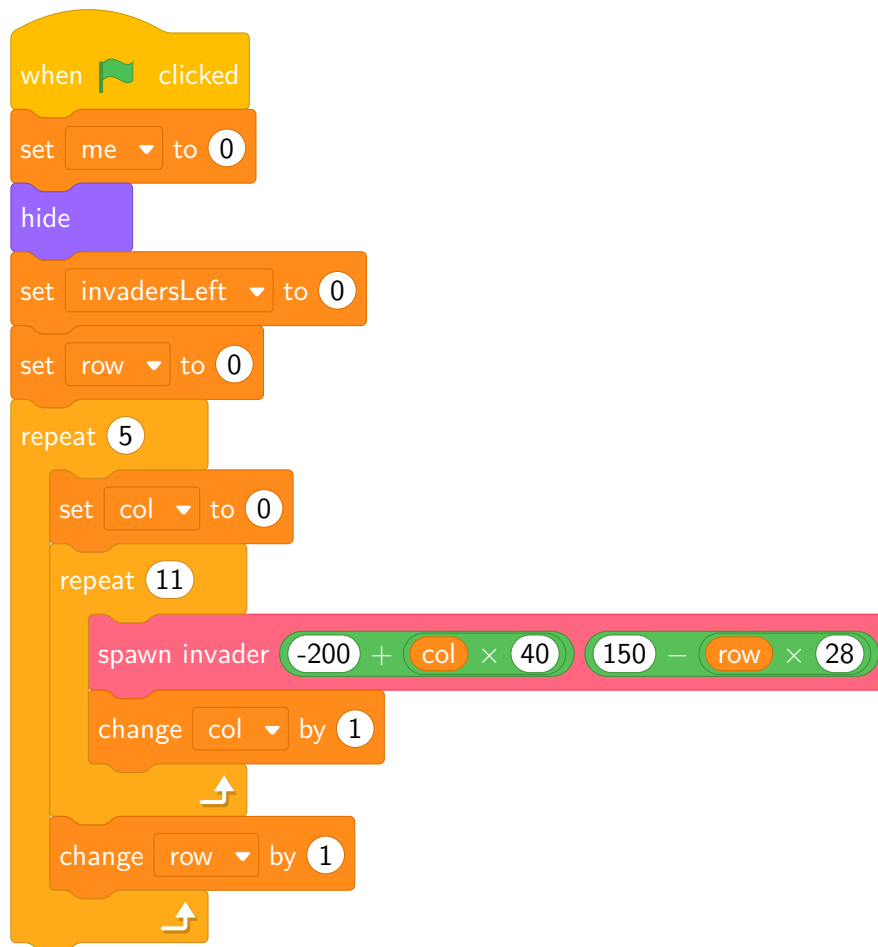
Build it.

1. Make the block: *My Blocks* -> *Make a Block*, name it **spawn invader**, and add two number inputs `x` and `y`. Under the `define` header, add `go to x: (x) y: (y)`, `create clone of [myself]`, `change [invadersLeft] by (1)`.
2. On the same sprite, **when green flag clicked**: `set [me] to (0)` (this original is the conductor, not an invader), `hide`, `set [invadersLeft] to (0)` - then call `spawn invader (0) (150)` once.
3. Add the clone hat **when I start as a clone**: `set [me] to (1)` and `show`.
4. **Checkpoint** - green flag: ONE invader appears at the top. The stamp works.
5. Now a row. Replace the single call with: `set [col] to (0)`, then `repeat (11)` calling `spawn invader ((-200) + (col x 40)) (150)` and `change [col] by (1)`. Green flag: a row of 11.
6. Now the grid. Wrap the row code in `set [row] to (0)` and `repeat (5)`, change `y` to `(150) - (row x 28)`, and after the inner loop `change [row] by (1)`.
7. **Checkpoint** - green flag: 55 invaders in formation.

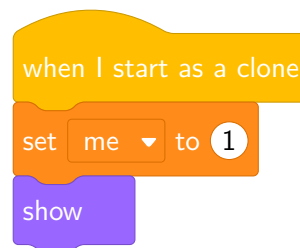
The custom block (left), the grid builder (middle), and each clone waking up (right):



My Block: stamp one invader, count it



Conductor: build a 5x11 grid



Each clone: I am a real invader

A For this sprite only variable like `me` is copied into each clone and then owned by it - 55 separate copies. The hidden original keeps `me = 0`; every clone sets its own to 1. That one line is how the fleet's brain tells itself apart from its body.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Rank the rows by colour: in `spawn invader`, before cloning, `switch costume` by `row` - row 0 `invader_green`, row 4 `invader_yellow`, etc.

Expected: Each rank looks different, like the real game.

2. Change the formation: try `repeat (4)` rows of `repeat (8)` for a smaller, faster wave.

Expected: A different-sized fleet builds cleanly - the loops do the work.

Project 5: March!

The fleet stands still - make it move as a single organism. A hidden conductor beats a `broadcast [march]`, and every clone steps once per beat. This version only marches one way - watch what happens when it meets the wall.

Get ready. No new sprites. Make variables `invaderDir` and `invaderStep`, both *For all sprites*.

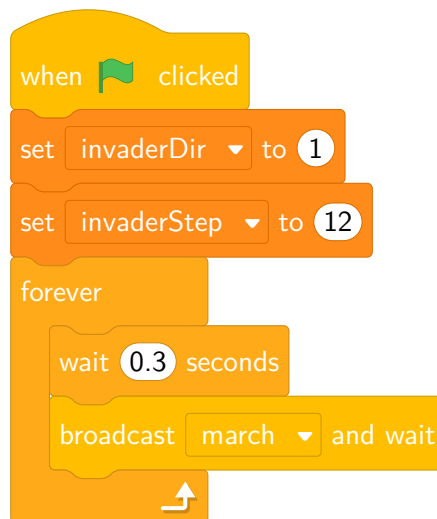
New blocks.

<code>broadcast [march] and wait</code>	Shout the beat and pause until every clone has finished its step - that is what keeps them in lockstep.
<code>when I receive [march]</code>	Each clone's marching orders: one step per beat, guarded by <code>me</code> so the hidden conductor stays put.

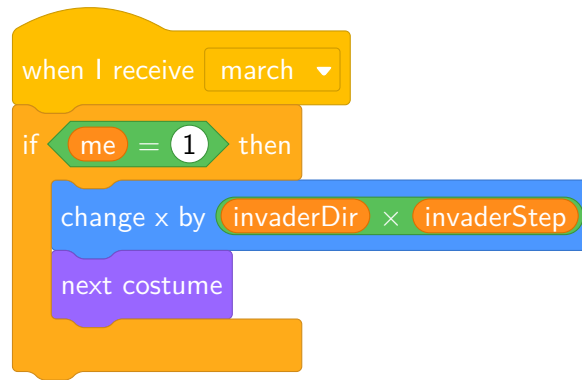
Build it.

1. In the conductor's `when green flag clicked` script, next to the other `set` blocks, add `set [invaderDir] to (1)` and `set [invaderStep] to (12)`.
2. At the bottom of the same script (under the grid loops), add a `forever`: `wait (0.3) seconds`, then `broadcast [march] and wait`.
3. On the Invader, add `when I receive [march]`. Guard everything with `if <(me) = (1)>`. Inside: `change x by the amount (invaderDir) x (invaderStep)`, then `next costume`.
4. **Checkpoint** - green flag: the fleet marches right in perfect lockstep, wiggling as it goes... and then grinds into the right wall and jams there. Half-built, as planned - the bounce is next.

Add the beat to the conductor (left); every invader obeys it (right):



Conductor: sets at the top, beat under the grid loops



Every invader: step and wiggle on the beat

Why a broadcast beat and not a `forever` loop on each clone? 55 independent loops drift apart; one conductor with `broadcast and wait` keeps all 55 perfectly in step. One brain, many bodies.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. March feel: lower `invaderStep` to (8) for finer steps, or raise it to (16) for lurching jumps.

Expected: The march feel changes with one number.

Project 6: Bounce and drop

Un-jam the fleet: when ANY clone touches a wall, the whole fleet drops one row and reverses. One shared variable `hitEdge` is the message from the many to the one.

Get ready. Make the variable `hitEdge` For all sprites.

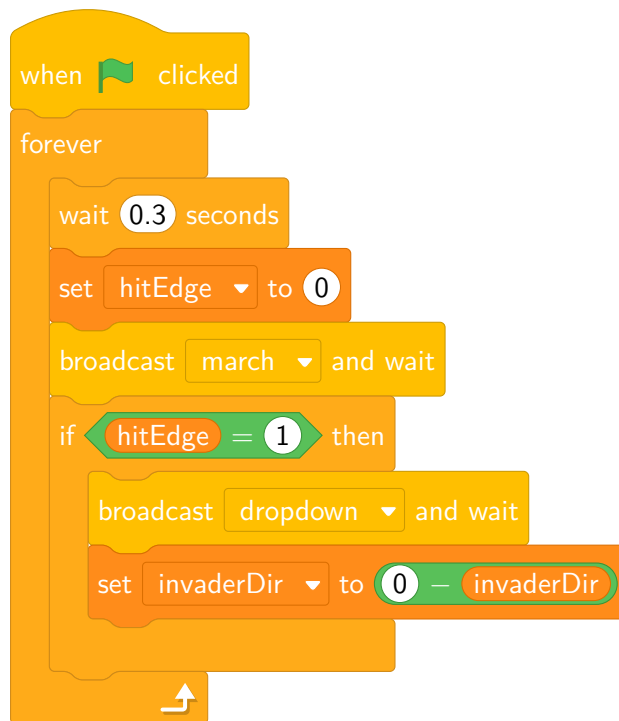
New blocks.

`(x position) x (invaderDir)` One test that catches BOTH walls: going right it means $x > 215$; going left, $x < -215$.

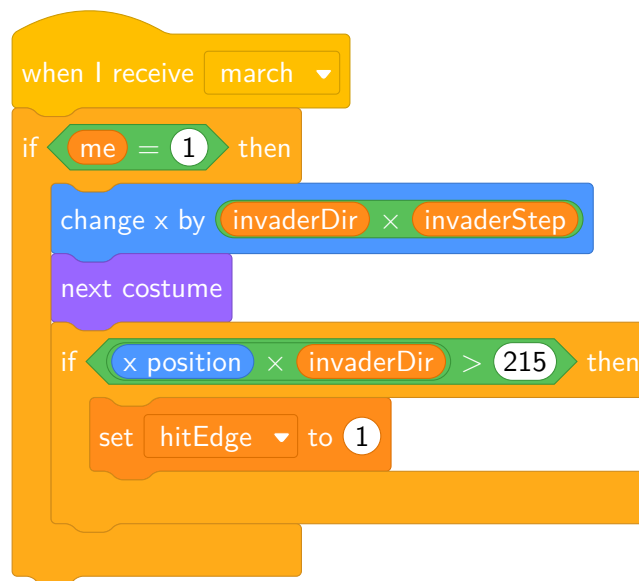
Build it.

1. In the `when I receive [march]` script, under `next costume`, add `if <((x position) x (invaderDir)) > (215)> then set [hitEdge] to (1)`.
2. In the conductor's beat loop: before the broadcast, `set [hitEdge] to (0)`. After it, `if <(hitEdge) = (1)> then broadcast [dropdown] and wait and set [invaderDir] to ((0) - (invaderDir))`.
3. On the Invader, add `when I receive [dropdown]`, also guarded by `me`: `change y by (-16)`.
4. **Checkpoint** - green flag: the fleet marches, drops at each wall, reverses, and slowly closes in. The invasion is on.

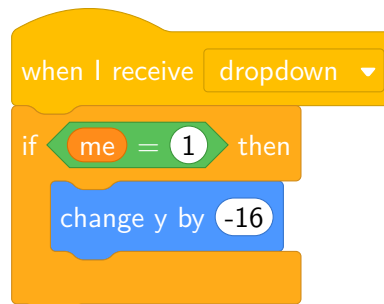
The conductor's finished heartbeat (left), and the two beats each invader obeys (right):



Conductor: the finished heartbeat



Every invader: step, wiggle, watch the wall



Every invader: drop a row

Any ONE clone setting `hitEdge` is enough - the conductor reverses the WHOLE fleet once, on the next beat. If instead each clone flipped `invaderDir` itself, all 55 would fight and jitter. Coordination through a shared variable is the lesson.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Bigger drops: change `change y by (-16)` to `(-24)` so the fleet closes in faster.

Expected: Each wall bounce brings them down harder.

Project 7: Shoot the fleet

The fleet marches and you can shoot - but lasers pass straight through. Wire up the hits: invaders die for points, and then ONE changed line gives you the most iconic difficulty curve in arcade history.

Get ready. Make a variable `score` For all sprites.

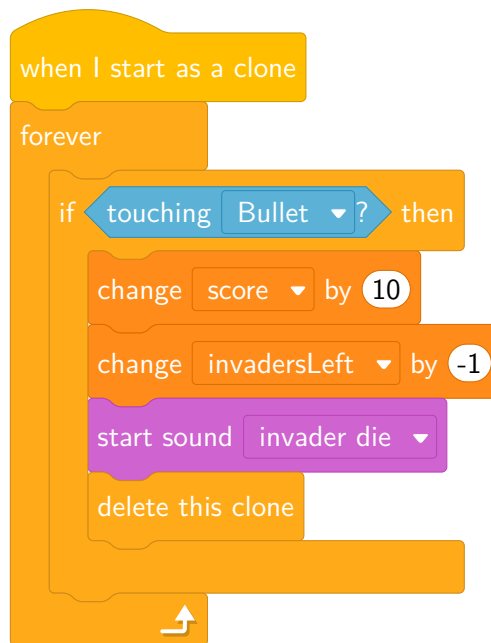
New blocks.

`touching [Bullet]?` Each invader watches for a laser; when hit it scores, counts down, and removes itself.

Build it.

1. On the **Player's** `when green flag clicked` script, add `set [score] to (0)`.
2. On the Invader, add a **SECOND** `when I start as a clone with a forever:` `if <touching [Bullet]>` then `change [score] by (10)`, `change [invadersLeft] by (-1)`, `start sound [invader die]`, `delete this clone`.
3. On the Bullet's clone loop, add a matching check: `if <touching [Invader]>` then `set [bulletActive] to (0)` and `delete this clone` - a hit reloads the gate too, not just the top of the screen.
4. **Checkpoint** - green flag: shoot the fleet. Invaders vanish, the score climbs... but the march never changes pace. Time for the famous one-liner:
5. In the conductor's heartbeat, replace `wait (0.3) seconds` with `wait ((0.05) + ((invadersLeft) / (120))) seconds`
6. Green flag: thin the fleet - the survivors sprint.

An invader taking a hit (left), and the Bullet's finished flight (right):



Each invader: hit by a laser -> score and vanish



Bullet: fly up, then clean up (finished)

The speed-up is the `wait ((0.05) + (invadersLeft / 120))`: 55 invaders wait about half a second between beats; the last one waits a fifth of that. In 1978 this was an accident - the hardware drew fewer sprites faster - and it became the best difficulty curve in games. You copied it on purpose, with one line.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Rank scoring: give each invader a *this sprite only* **rank** set from **row** at spawn, and score $(10) + ((4) - (\text{rank})) \times (10)$ so the back rows pay more.

Expected: The scariest back row is worth 50; the front row 10.

Project 8: Win and lose

The game plays - now let it END. Win by clearing the fleet; lose if it reaches you or lands. The win check reuses **invadersLeft** - the same count that drives the speed-up decides the victory.

Get ready. Add two text sprites: **GameOver** (text_game_over) and **YouWin** (text_you_win), both hidden at the start.

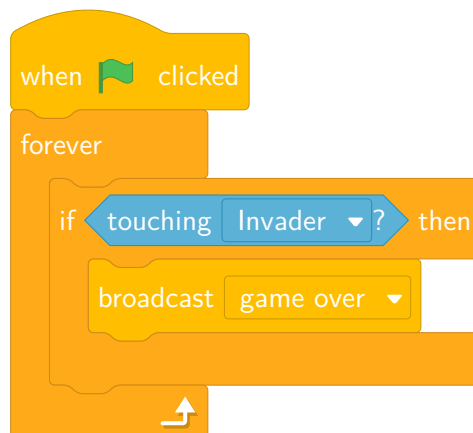
New blocks.

<code>(invadersLeft) = (0)</code>	The win check - when the count the spawner built up reaches zero, the fleet is gone.
<code>stop [all]</code>	Freeze the whole project on the banner - game over means game OVER.

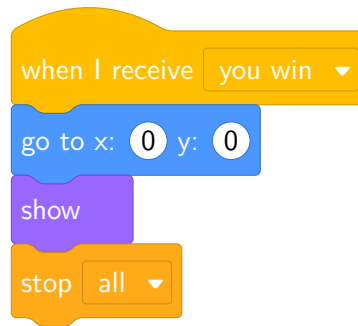
Build it.

1. On **GameOver** and **YouWin**: `when green flag clicked` `hide`; and `when I receive [game over] / [you win]` `go to x: (0) y: (0)`, `show`, `stop [all]`.
2. On the **Player**, add a `forever` that does `if <touching [Invader]??>` then `broadcast [game over]`.
3. In the Invader's `when I receive [dropdown]` script, after the drop, add `if <(y position) < (-120)>` then `broadcast [game over]` - the fleet has landed.
4. In the conductor's heartbeat, at the bottom of the loop, add `if <(invadersLeft) = (0)>` then `broadcast [you win]`.
5. **Checkpoint** - green flag, one last time: clear the fleet to win; let it touch you or land to lose. The game is COMPLETE.

The ship's loss check (left) and the win banner (right):



Player: touched by the fleet -> lose



YouWin: show the banner

Three different roads lead to `broadcast [game over]` - touching the ship, and the fleet landing - but ONE script shows the banner. Broadcasts collect all the ways to lose into a single ending.

Your task. Now make your project better. Do these in order. Each one builds on the last.

1. Return fire: add a **Bomb** sprite. In an invader's loop, rarely (`if <(pick random (1) to (400)) = (1)>`) `go to` itself and `create clone of [Bomb]`; the bomb clone falls until it hits the Player.

Expected: Now the fleet shoots back.

2. Bonus mothership: add a **UFO** (`ufo_purple`) that slides across the top every 20 seconds for bonus points if you tag it.

Expected: A high-value target crosses the sky - pure Dino-style world-slide.